

Análisis de Algoritmos 2025/2026

Práctica 2

Shaofan Xu, Alejandro Zheng (120/127)

Código	Gráficas	Memoria	Total

1. Introducción.

El objetivo de esta práctica es analizar experimentalmente el comportamiento y la eficiencia de los algoritmos de ordenación basados en la técnica de divide y vencerás, concretamente MergeSort y QuickSort.

Para ello, se implementarán ambos algoritmos en lenguaje C y se evaluará su rendimiento sobre tablas de distintos tamaños, midiendo tanto el tiempo de ejecución como el número de operaciones básicas realizadas. A partir de los datos experimentales obtenidos, se generarán gráficas que permitan comparar los resultados empíricos con el comportamiento teórico esperado, identificando las diferencias entre los casos mejor, peor y medio de cada algoritmo.

También se discutirán los resultados en términos de eficiencia temporal y se analizará la influencia de diferentes estrategias de selección del pivote en el rendimiento del algoritmo QuickSort.

2. Objetivos

2.1 Apartado 1

El objetivo de este apartado es implementar correctamente el algoritmo de ordenación MergeSort, utilizando la técnica de divide y vencerás y aplicando una función auxiliar de combinación (merge, es decir, la función combinar que hemos aprendido en la clase).

Y luego verificamos su correcto funcionamiento mediante el programa exercise4.c reutilizado de la práctica anterior, así comprobando que las tablas se ordenan correctamente.

2.2 Apartado 2

El objetivo de este apartado es evaluar experimentalmente el rendimiento del algoritmo MergeSort, midiendo el tiempo de ejecución y el número de operaciones básicas para distintos tamaños de entrada.

Usamos el programa exercise5.c para ellos. Finalmente, se compararán los resultados empíricos con el comportamiento teórico del algoritmo ($O(n \log n)$), generando gráficas.

2.3 Apartado 3

El objetivo de este apartado es implementar el algoritmo de ordenación QuickSort, también basado en la estrategia de divide y vencerás, utilizando inicialmente el primer elemento como pivote. Se desarrollarán las funciones partition() y median().

Y verificamos la correcta ordenación de las tablas mediante exercise4.c.

2.4 Apartado 4

En este apartado, el objetivo es analizar empíricamente el comportamiento del algoritmo QuickSort. Se registrarán el tiempo de ejecución y las operaciones básicas promedio, máximas y mínimas para diferentes tamaños de tabla, utilizando el programa exercise5.c. Los resultados se compararán con el comportamiento teórico del algoritmo, cuyo caso medio es $O(n \log n)$ y peor caso $O(n^2)$.

2.5 Apartado 5

El objetivo de este apartado es comparar distintas estrategias de selección del pivote en QuickSort y analizar su efecto sobre el rendimiento. Se implementarán las funciones `median_avg()` y `median_stat()` para elegir el pivote central y el pivote basado en la “mediana de tres”, respectivamente.

Posteriormente, se evaluará y comparará el tiempo de ejecución y el número de operaciones básicas obtenidos con cada estrategia (`median`, `median_avg` y `median_stat`), comparando qué método ofrece una mejor eficiencia.

3. Herramientas y metodología

Para el desarrollo de esta práctica se ha utilizado un entorno de desarrollo basado en Linux (Ubuntu 24.04.3), aprovechando las herramientas de la línea de comandos que ofrece este sistema.

Las herramientas principales empleadas han sido:

- **Editor de Código:** Visual Studio Code.
- **Compilador:** gcc, utilizando los flags `-Wall` para activar todas las advertencias, `-pedantic` para generar la información necesaria para la depuración, `-ansi` para usar el standard C89. Y, además `-O3` para optimizar el programa.
- **Análisis de Memoria:** Valgrind, una herramienta fundamental para detectar fugas de memoria y accesos a memoria inválidos.
- **Automatización:** make, mediante un fichero Makefile, para automatizar el proceso de compilación y ejecución de los distintos ejercicios.
- **Visualización de Datos:** Gnuplot, para generar las gráficas a partir de los ficheros de resultados y poder realizar un análisis visual comparativo del rendimiento de los algoritmos.

3.1 Apartado 1

```
1. int merge(int* tabla, int ip, int iu, int imedio)
2. int mergesort(int* tabla, int ip, int iu)
```

Para MergeSort necesitamos implementar la función de merge que hace la unión de las tablas después de dividirla. Luego, en la implementación de mergesort incluimos es función de merge.

3.2 Apartado 2

Para este apartado de comprobar el correcto funcionamiento usamos el `exercise5.c` para MergeSort de esta forma nos guarda el resultado de ejecución en un fichero que indicamos nosotros.

3.3 Apartado 3

```
1. int partition(int* tabla, int ip, int iu, int *pos)
2. int quicksort(int* tabla, int ip, int iu)
3. int median(int *tabla, int ip, int iu, int *pos)
```

Para el **QuickSort** necesitamos implementar primero la función **partition** que se ordenará la table basando en la elección de pivote, donde los elementos más pequeños que el pivote irán a la izquierda del pivote y los más grandes a la derecha del pivote.

En este caso, la selección de pivote hacemos mediante la función **median** donde se elegirá al primer elemento de la tabla como el pivote. Y luego, en la función **quicksort** llamamos a la **partition** para ordenar toda la tabla.

3.4 Apartado 4

En este caso, para verificar el correcto funcionamiento basta usar el `exercise5.c` para **QuickSort** y así nos guarda su resultado de ejecución en un fichero que indicamos nosotros.

3.5 Apartado 5

```
1. int median_avg(int *tabla, int ip, int iu, int *pos)
2. int median_stat(int *tabla, int ip, int iu, int *pos)
```

Para implementar la función **median_avg** simplemente elegiremos el pivote como el elemento que está en el medio de la tabla. Luego para **median_stat** elegimos el pivote como el elemento mediano de la tabla de las siguientes posiciones: `ip`, `mid` e `iu`, pues hemos hecho como una especie de árbol de decisión usando sentencias `if/else`.

4. Código fuente

Aquí ponéis el código fuente **exclusivamente de las rutinas que habéis desarrollado vosotros** en cada apartado.

4.1 Apartado 1

```
1. int merge(int *tabla, int ip, int iu, int imedio)
2. {
3.
4.     int *aux_table = NULL;
5.     int size;
6.     int i, j, k;
7.     int ob = 0;
8.
9.     /*asserting contidions*/
10.    assert(tabla != NULL);
11.    assert(ip <= iu);
12.    assert(ip <= imedio);
```

```

13.  assert(iu >= imedio);
14.
15.  /*Calculate the size of table*/
16.  size = (iu - ip) + 1;
17.
18.  /*malloc an auxiliar table */
19.  aux_table = malloc(sizeof(int) * size);
20.  if (aux_table == NULL)
21.  {
22.      return ERR;
23.  }
24.
25.  i = ip;
26.  j = imedio + 1;
27.  k = 0;
28.
29.  /*process of combining*/
30.  while (i <= imedio && j <= iu)
31.  {
32.      if (tabla[i] < tabla[j])
33.      {
34.          aux_table[k] = tabla[i];
35.          ob++;
36.          i++;
37.          k++;
38.      }
39.      else
40.      {
41.          aux_table[k] = tabla[j];
42.          ob++;
43.          j++;
44.          k++;
45.      }
46.  }
47.
48.  /*copy the rest elements of aux_table*/
49.  if (i > imedio)
50.  {
51.      for (; j <= iu; j++, k++)
52.      {
53.          aux_table[k] = tabla[j];
54.      }
55.  }
56.
57.  /*copy the rest elements of aux_table*/
58.  if (j > iu)
59.  {
60.      for (; i <= imedio; i++, k++)
61.      {
62.          aux_table[k] = tabla[i];
63.      }
64.  }
65.
66.  /*copy the aux_table to table*/
67.  for (i = ip, j = 0; i <= iu; i++, j++)
68.  {
69.      tabla[i] = aux_table[j];
70.  }
71.
72.  /*free aux_table*/
73.  free(aux_table);
74.
75.  return ob;
76. }
77.

```

```

1.  int mergesort(int *tabla, int ip, int iu)
2.  {
3.      int imedio = 0;

```

```

4.  int ob1, ob2, ob3, ob_total;
5.
6.  /*asserting conditions*/
7.  assert(tabla != NULL);
8.  assert(ip <= iu);
9.
10. /*table with one element*/
11. if (ip == iu)
12.     return OK;
13.
14. /*calculate the middle*/
15. imedio = (iu + ip) / 2;
16.
17. /*dividing the table*/
18. ob1 = mergesort(tabla, ip, imedio);
19. if (ob1 == ERR)
20. {
21.     return ERR;
22. }
23.
24. ob2 = mergesort(tabla, imedio + 1, iu);
25. if (ob2 == ERR)
26. {
27.     return ERR;
28. }
29.
30. /*combine the tabl*/
31. ob3 = merge(tabla, ip, iu, imedio);
32. if (ob3 == ERR)
33. {
34.     return ERR;
35. }
36.
37. ob_total = ob1 + ob2 + ob3;
38.
39. return ob_total;
40. }
41.

```

4.3 Apartado 3

```

1. int median(int *tabla, int ip, int iu, int *pos)
2. {
3.     assert(tabla!=NULL);
4.     assert(ip>=0);
5.     assert(ip<=iu);
6.     assert(pos!=NULL);
7.
8.     *pos=ip;
9.     return 0;
10. }
11.

```

```

1. int partition(int* tabla, int ip, int iu, int *pos)
2. {
3.     int element,i,obs=0,p_obs;
4.
5.     assert(tabla!=NULL);
6.     assert(ip>=0);
7.     assert(ip<=iu);
8.     assert(pos!=NULL);
9.

```

```

10. if((p_obs=median(tabla,ip,iu,pos))==ERR)return ERR;
11. element=tabla[*pos];
12. obs+=p_obs;
13.
14. swap(&tabla[ip],&tabla[*pos]);
15. *pos=ip;
16.
17. for(i=ip+1;i<=iu;i++)
18. {
19.     obs++;
20.     if(tabla[i]<element)
21.     {
22.         (*pos)++;
23.         swap(&tabla[i],&tabla[*pos]);
24.     }
25. }
26. swap(&tabla[ip],&tabla[*pos]);
27. return obs;
28. }

```

```

1. int quicksort(int* tabla, int ip, int iu)
2. {
3.     int position=0,obs=0;
4.     assert(tabla!=NULL);
5.     assert(ip>=0);
6.     assert(ip<=iu);
7.
8.     /*Base case*/
9.     if(ip==iu)return 0;
10.    else{
11.        if((obs=partition(tabla,ip,iu,&position))==ERR) return ERR;
12.        if(ip<position-1)obs+=quicksort(tabla,ip,position-1);
13.        if(position+1<iu)obs+=quicksort(tabla,position+1,iu);
14.    }
15.    return obs;
16. }

```

4.5 Apartado 5

```

1. int median_avg(int *tabla, int ip, int iu, int *pos)
2. {
3.     assert(tabla!=NULL);
4.     assert(ip>=0);
5.     assert(ip<=iu);
6.     assert(pos!=NULL);
7.
8.     *pos=(ip+iu)/2;
9.     return 0;
10. }

```

```

1. int median_stat(int *tabla, int ip, int iu, int *pos)
2. {
3.     int mid,obs;
4.     assert(tabla != NULL);
5.     assert(pos != NULL);
6.     assert(ip >= 0);
7.     assert(ip <= iu);
8.
9.     mid= (ip + iu) / 2;
10.    obs=0;
11.    if (tabla[ip] <= tabla[mid]) {
12.        if (tabla[mid] <= tabla[iu]) {
13.            *pos = mid;
14.        } else {
15.            if (tabla[ip] <= tabla[iu]) {
16.                *pos = iu;

```

```

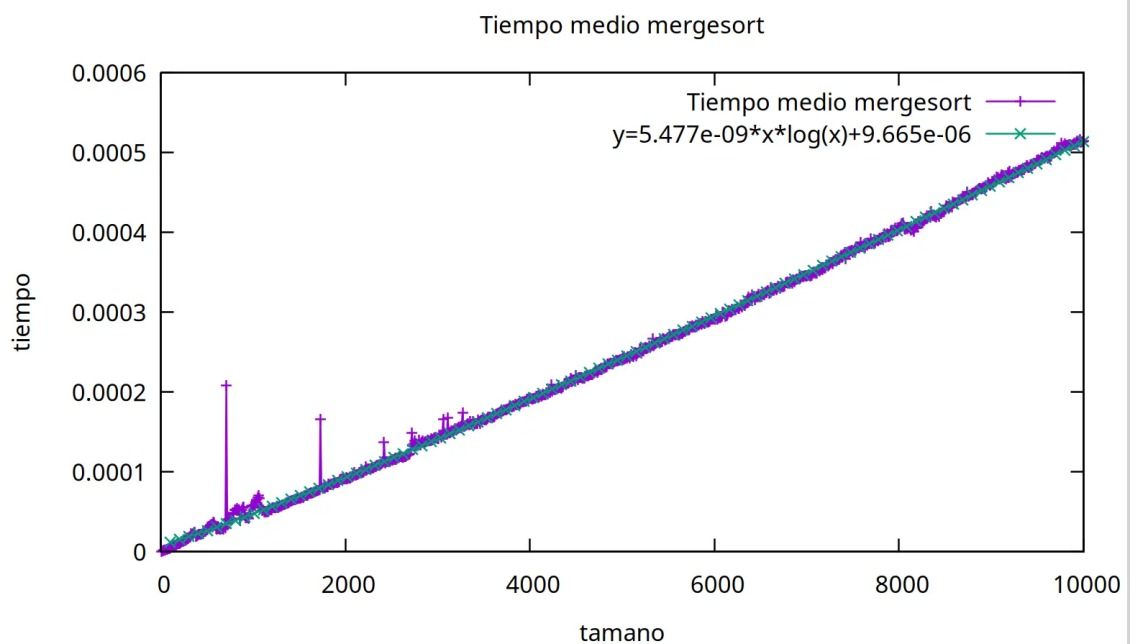
17.         } else {
18.             *pos = ip;
19.         }
20.     }
21. } else {
22.     if (tabla[ip] <= tabla[iu]) {
23.         *pos = ip;
24.     } else {
25.         if (tabla[mid] <= tabla[iu]) {
26.             *pos = iu;
27.         } else {
28.             *pos = mid;
29.         }
30.     }
31. }
32. obs+=3;
33. return obs;
34. }

```

5. Resultados, Gráficas

5.1 Apartado 1

Gráfica de tiempo medio de MergeSort



En esta gráfica muestra el tiempo medio de MergeSort con un ajuste de $n*\log(n)$ y observamos que la ecuación ajustada coincide con la gráfica de tiempo medio de MergeSort. Por lo tanto, podemos concluir que el rendimiento medio de MergeSort es $O(N*\log(N))$. La siguiente imagen es el ajuste de la ecuación.


```

gnuplot> f(x)=a*x*log(x)+b
gnuplot> fit f(x) "exercice5_mergesort_median_avg.log" u 1:2 via a,b
iter    chisq      delta/lim  lambda    a          b
0 2.6347424218e+12  0.00e+00  3.63e+04  1.000000e+00 1.000000e+00
1 6.5830595133e+05 -4.00e+11  3.63e+03  4.832165e-04 9.999832e-01
2 2.7852624857e+02 -2.36e+08  3.63e+02 -1.653904e-05 9.999621e-01
3 2.7735205415e+02 -4.23e+02  3.63e+01 -1.650663e-05 9.978521e-01
4 1.8898108018e+02 -4.68e+04  3.63e+00 -1.362452e-05 8.236838e-01
5 3.8535028615e-01 -4.89e+07  3.63e-01 -6.100036e-07 3.720377e-02
6 1.3828564795e-07 -2.79e+11  3.63e-02  5.186286e-09 2.724691e-05
7 5.2182762296e-08 -1.65e+05  3.63e-03  5.477219e-09 9.665529e-06
8 5.2182762294e-08 -3.69e-06  3.63e-04  5.477220e-09 9.665446e-06
iter    chisq      delta/lim  lambda    a          b

After 8 iterations the fit converged.
final sum of squares of residuals : 5.21828e-08
rel. change during last iteration : -3.6903e-11

degrees of freedom (FIT_NDF) : 998
rms of residuals (FIT_STDFIT) = sqrt(WSSR/ndf) : 7.231e-06
variance of residuals (reduced chisquare) = WSSR/ndf : 5.22873e-11

Final set of parameters      Asymptotic Standard Error
=====
a = 5.47722e-09 +/- 8.441e-12 (0.1541%)
b = 9.66545e-06 +/- 4.333e-07 (4.483%)

correlation matrix of the fit parameters:
      a      b
a      1.000
b     -0.849  1.000

```

Gráfica de tiempo mejor de MergeSort

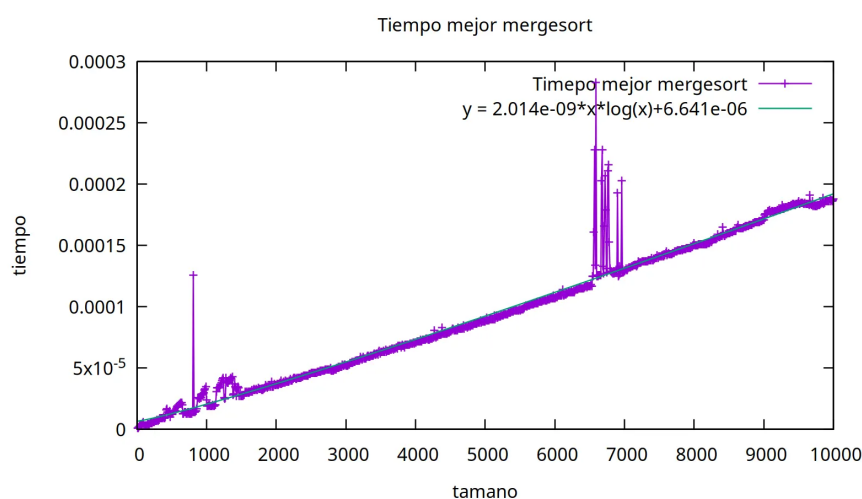
Para esta gráfica de tiempo mejor de MergeSort hemos utilizado una ecuación de $f(x)=a*x*\log(x)+b$ para ajusta la gráfica. La siguiente imagen es el resultado que obtenemos a ajusta usando Gnuplot. Asimismo, vemos que ecuación ajustada en la gráfica se coincide con la línea de tiempo mejor de MergeSort. En consecuencia, podemos concluir que el tiempo mejor de MergeSort es $O(N*\log(N))$.

```

Final set of parameters      Asymptotic Standard Error
=====
a = 2.01433e-09 +/- 1.255e-11 (0.623%)
b = 6.64165e-06 +/- 6.442e-07 (9.699%)

correlation matrix of the fit parameters:
      a      b
a      1.000
b     -0.849  1.000

```

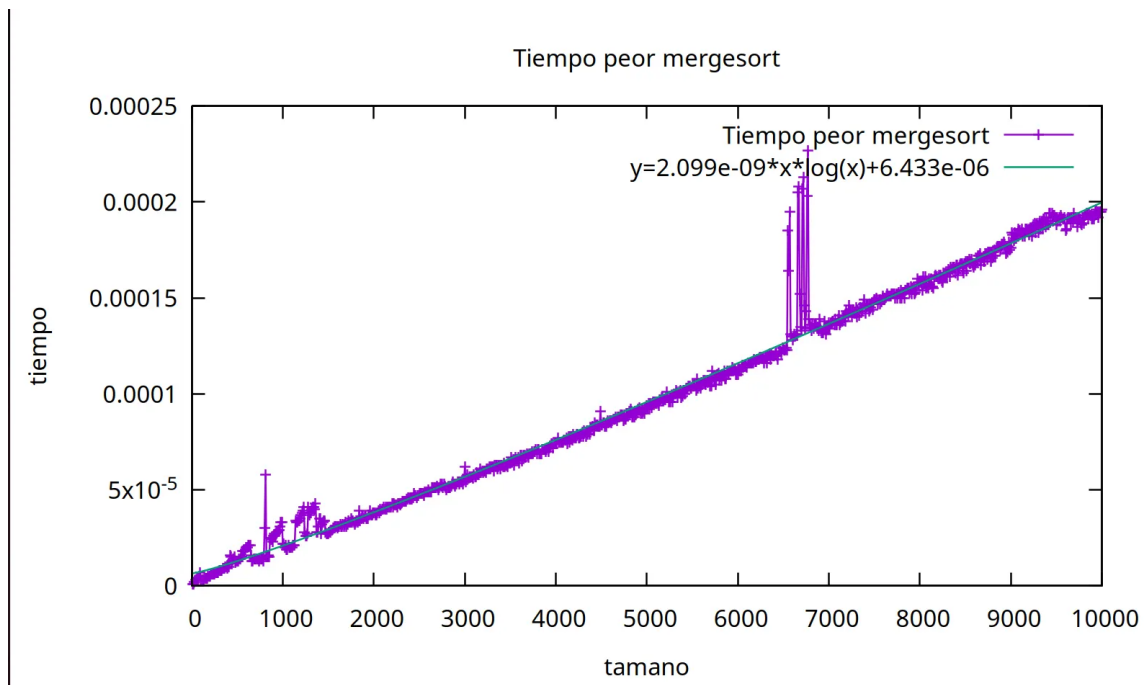


Gráfica de tiempo peor de MergeSort

Para el tiempo peor de MergeSort hacemos de forma similar, con una ecuación de $f(x)=a*x*\log(x)+b$, intentamos ajustar la gráfica con Gnuplot. El resultado que obtenemos con un erro pequeño y observado la gráfica siguiente vemos que coincide las líneas de tiempo peor y la ecuación que hemos propuesto. Asimismo, concluimos que el tiempo peor de MergeSort también es $O(N*\log(N))$.

```
Final set of parameters          Asymptotic Standard Error
=====
a          = 2.09955e-09        +/- 9.141e-12   (0.4354%)
b          = 6.43313e-06        +/- 4.692e-07   (7.293%)

correlation matrix of the fit parameters:
      a      b
a      1.000
b     -0.849 1.000
```



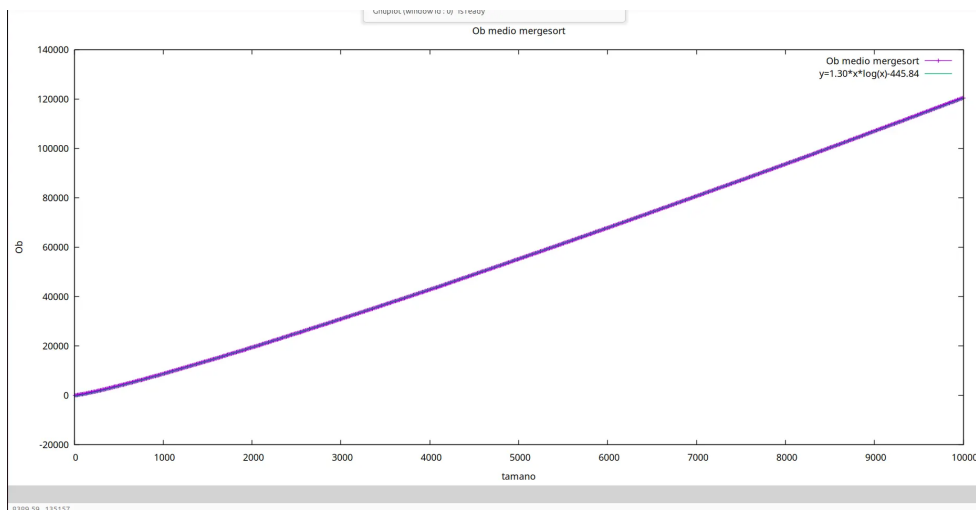
5.2 Apartado 2

En este apartado vamos a representar las gráficas en tiempo medio, pero y mejor en OB de Mergesort:

Para tiempo medio en OB de MergeSort

La siguiente gráfica de tiempo medio en Ob de MergeSort con un ajuste de $N*\log(N)$ vemos que la gráfica de OB coincide con la ecuación ajustada. El resultado que obtenemos usando OB coherente con el resultado de MergeSort midiendo tiempo.

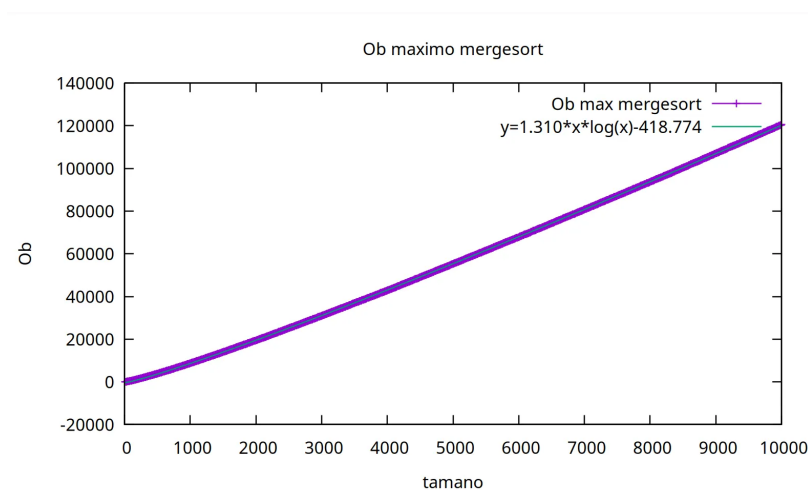
Final set of parameters		Asymptotic Standard Error	
=====		=====	
a	= 1.30956	+/- 0.0001545	(0.0118%)
b	= -445.84	+/- 7.929	(1.779%)
correlation matrix of the fit parameters:			
	a	b	
a	1.000		
b	-0.849	1.000	



Para tiempo peor en OB de MergeSort

La siguiente gráfica de tiempo peor en Ob de MergeSort con un ajuste de $N \cdot \log(N)$ vemos que la gráfica de OB coincide con la ecuación ajustada. Asimismo, podemos asegurar por segunda vez usando la gráfica de OB, determinamos que el tiempo peor de MergeSort es $O(N \cdot \log(N))$.

Final set of parameters		Asymptotic Standard Error	
=====		=====	
a	= 1.31029	+/- 0.0001508	(0.01151%)
b	= -418.774	+/- 7.742	(1.849%)
correlation matrix of the fit parameters:			
	a	b	
a	1.000		
b	-0.849	1.000	

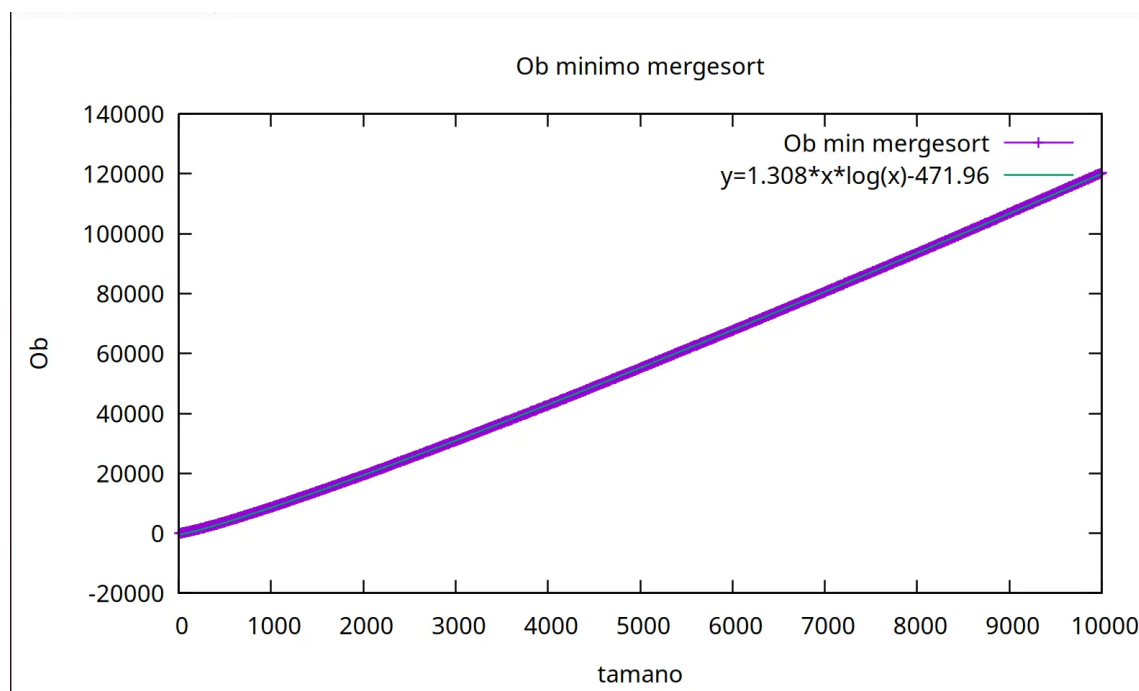


Para tiempo mejor en OB de MergeSort:

De forma similar a tiempo medio en OB de MergeSort, aplicamos para el tiempo mejor en OB. Usamos una ecuación logarítmica de $f(x)=a*x*\log(x)+b$ para ajustar la gráfica de OB min MergeSort. El resultado que obtenemos es coherente con conclusión de que tiempo mejor de MergeSort es $O(N*\log(N))$ puesto que el error de ajuste es menor y en la representación de gráfica vemos es solapa las dos líneas.

```
Final set of parameters      Asymptotic Standard Error
=====
c      = 1.30881             +/- 0.0001612      (0.01232%)
d      = -471.96             +/- 8.276          (1.753%)

correlation matrix of the fit parameters:
      c      d
c      1.000
d     -0.849 1.000
```



5.3 Apartado 3

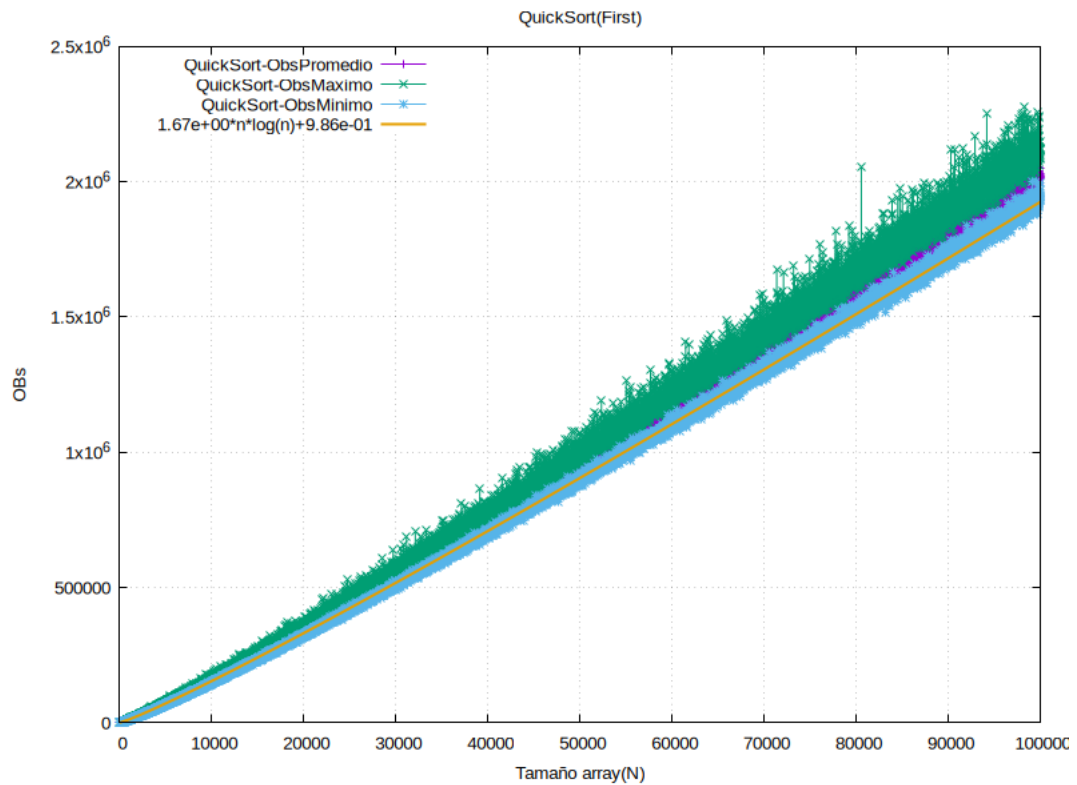
En este caso, para el apartado 3 no tenemos que hacer ninguna gráfica, ya que es simplemente comprobar el correcto funcionamiento de la función **Quicksort** implementada.

1. Running exercise4_quicksort
2. Practice number 1, section 4
3. Done by: Shaofan Xu y Alejandro Zheng
4. Grupo: 120/127
5. 1 2 3 4 5

5.4 Apartado 4

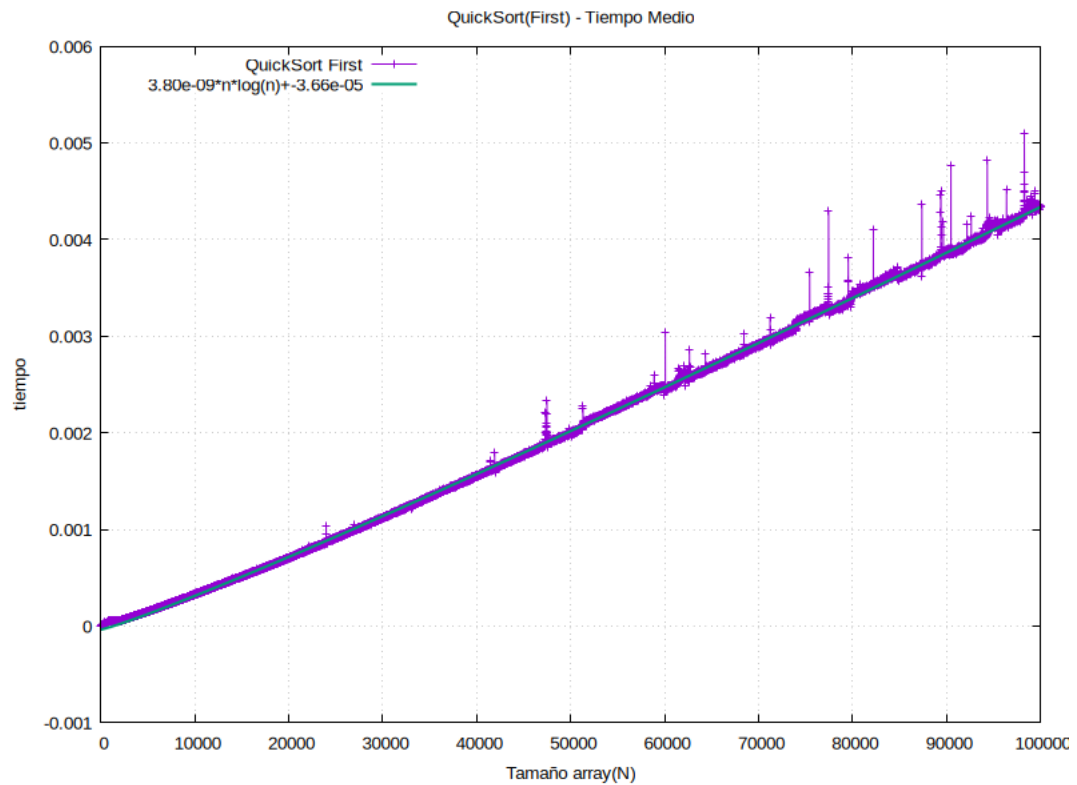
Resultados del apartado 4.

Gráfica comparando los tiempos mejor peor y medio en OBs para QuickSort, comentarios a la gráfica.



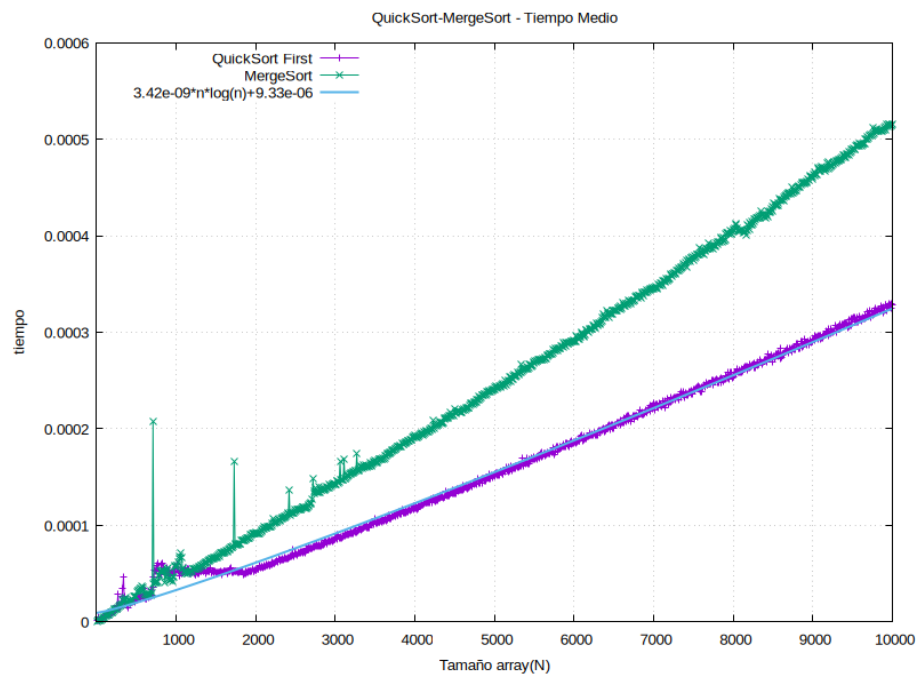
Para esta gráfica, obtenemos un resultado bastante trivial, ya que podemos ver que la gráfica de ObsMaximo tiene un rendimiento peor en OBs que ObsPromedio y ObsMinimo (es la definición de máximo, mínimo y promedio). Además, podemos ver que al ajustar a una función $ax \log + b$ se obtiene un error razonable, lo cual su rendimiento es $O(N \log N)$.

Gráfica con el tiempo medio de reloj para QuickSort, comentarios a la gráfica.



En esta gráfica, podemos observar que el rendimiento de QuickSort en el caso medio es $O(N \log N)$ con la ayuda de ajuste de datos a una función $a \log x + b$ y en este caso se aproxima bastante bien, lo cual, verificó su rendimiento teórico.

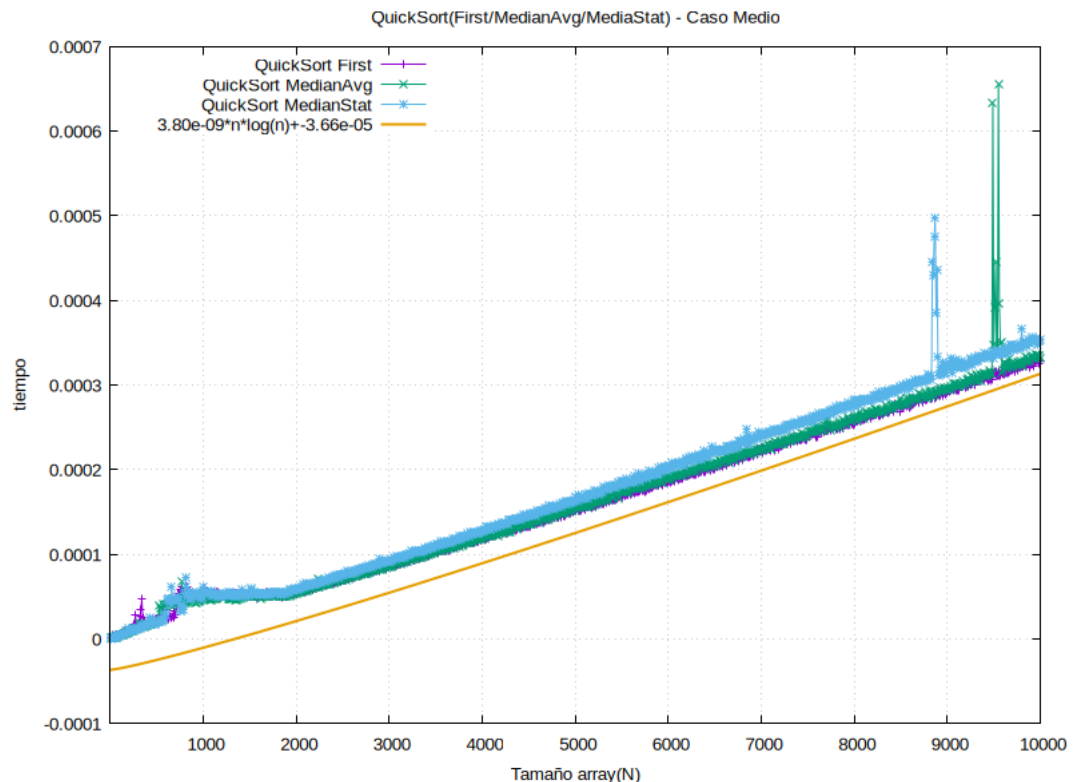
Gráfica comparando el tiempo medio de reloj de MergeSort y QuickSort



En esta gráfica de tiempo medio de reloj, observamos que QuickSort funciona más rápido que MergeSort, lo cual es razonable, ya que QuickSort no necesita reservar memoria y liberar memoria constantemente. Y luego con el ajuste, comprobamos que en definitiva los 2 algoritmos tienen un comportamiento $O(N \log N)$ en el caso medio.

5.5 Apartado 5

Gráfica comparando el tiempo medio de reloj de Quicksort usando los pivotes median, median_avg y median_stat.



En este caso, comparamos las tres estrategias de elegir pivote de QuickSort: primer elemento, el medio, y la mediana entre el primero, el medio y el último elemento. Lo cual, observamos que, en este caso, el elegir el primer elemento pivote es el más eficiente, y con un rendimiento similar si cogemos la intermedia. Sin embargo, al elegir la “mediana” es el caso peor, ya que en este caso no es la mediana de toda la tabla, sino de 3 elementos, además, esto suma con las 3 operaciones básicas extra que lleva la función **median_stat**. Pero en los 3 casos tienen un comportamiento $O(N \log N)$ en el caso medio (ver el ajuste).

5. Respuesta a las preguntas teóricas.

5.1 Compara el rendimiento empírico de los algoritmos con el caso medio teórico en cada caso. Si las trazas de las gráficas del rendimiento son muy picudas razonad porqué ocurre esto.

Para los dos algoritmos el rendimiento empírico en el caso medio es $O(N \cdot \log(N))$, lo podemos observar mediante la gráfica anteriores. Si comparamos con la teoría nos coincide con la práctica. En la gráfica de tiempo medio de reloj de los dos algoritmos se ajustaba bastante bien con una ecuación de $f(x) = a \cdot x \cdot \log(x) + b$. Por lo tanto, hemos llevado de forma correcta la teoría en la práctica.

Las gráficas aparecen algunas picudas es debido que el tiempo de ejecución no solo dependen del algoritmo en sí, sino también existen pequeñas variaciones en la parte de hardware de ordenador, gestiones de memoria.

5.2 Razonad el resultado obtenido al comparar las versiones de quicksort con los diferentes pivotes tanto si se obtienen diferencias apreciables como si no.

Aunque todos sus rendimientos son $O(N \log N)$, sin embargo, existe unas pequeñas diferencias entre las 3 estrategias de elegir pivote, en nuestro caso, nuestra conclusión es que al elegir el pivote el elemento mediano de la tabla con índices entre ip , mid , iu , tiene un peor rendimiento, ya que hacen operaciones adicionales de comparación. Y luego el first-pivote es el más rápido de todo, y enseguida esta quicksort con pivote intermedio. Entre QuickSort con primer elemento como pivote y elemento intermedio como pivote tienen una diferencia apreciable, ya que ambos no se realizan comparaciones extras, pero QuickSort con `median_stat` sí.

5.3 ¿Cuáles son los casos mejor y peor para cada uno de los algoritmos? ¿Qué habrá que modificar en la práctica para calcular estrictamente cada uno de los casos (también el caso medio)?

En el MergeSort, el caso mejor y caso peor son $O(N \cdot \log(N))$, hemos comprobado con las gráficas anteriores. Para medir el caso mejor de mergeSort la permutación que introducimos para ordenar que tiene que está ordenada previamente de menor a mayor y para el caso peor sería permutación ordenado de mayor a menor. Para el caso medio sería un arreglo con elementos aleatorios como hemos hecho en la práctica.

En el QuickSort (First-pivote), el caso mejor es $O(N \log N)$, y se obtiene si cuando partimos podemos “dividir” el array casi la mitad, es decir el número de los elementos menor que pivote es casi lo mismo que el número de los elementos mayor que el pivote. Y luego el caso peor, si ordenamos un array ordenado. Pues para el caso peor habría que pedir ordenar una tabla ordenada, y el caso mejor, pues que el elemento mediano de la tabla está en la primera posición siempre.

5.4 ¿Cuál de los dos algoritmos estudiados es más eficiente empíricamente? Compara este resultado con la predicción teórica.

Si observamos la gráfica de comparación de los dos algoritmos, obtenemos el resultado de que el QuickSort es más eficiente empíricamente que MergeSort para el caso medio. Si comparamos con la predicción teórica, los dos algoritmos tienen un rendimiento igual de $O(N \cdot \log(N))$. Para peor caso, MergeSort es más eficiente que QuickSort ya que el rendimiento peor de QuickSort es $O(N^2)$.

5.5 ¿Cual(es) de los algoritmos es/son más eficientes desde el punto de vista de la gestión de memoria? Razona este resultado.

El algoritmo de QuickSort es más eficiente desde el punto de vista de la gestión de memoria ya que para MergeSort hay que crear un array adicional. Sin embargo, para QuickSort se ordena in-place sin crear un array adicional. Por lo tanto, desde punto de vista de memoria, QuickSort es más eficiente.

6. Conclusiones finales.

En conclusión, esta práctica hemos hecho la medición de rendimiento de los dos algoritmos de clase de divide y vencerás. Nos permitió experimental de forma empírica el rendimiento de los dos algoritmos en diferentes casos y los resultados que obtenemos son coherente con la teoría. Asimismo, hemos experimentado las diferentes versiones de QuickSort con los diferentes pivotes. Además, hemos obtenido la conclusión de que el QuickSort es más eficiente que MergeSort desde punto de vista de gestión de memoria puesto que MergeSort requiere memoria adicional para ordenar los elementos.